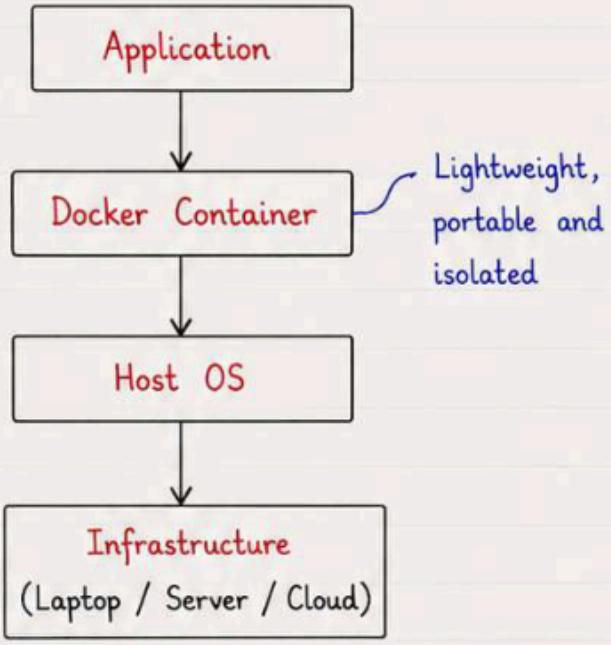
Docker Notes

1. What is Docker ?

Docker is an open-source platform that helps us to build, ship and run applications inside containers. It allows an application to run consistently on any environment.



2. Why Docker was created ?

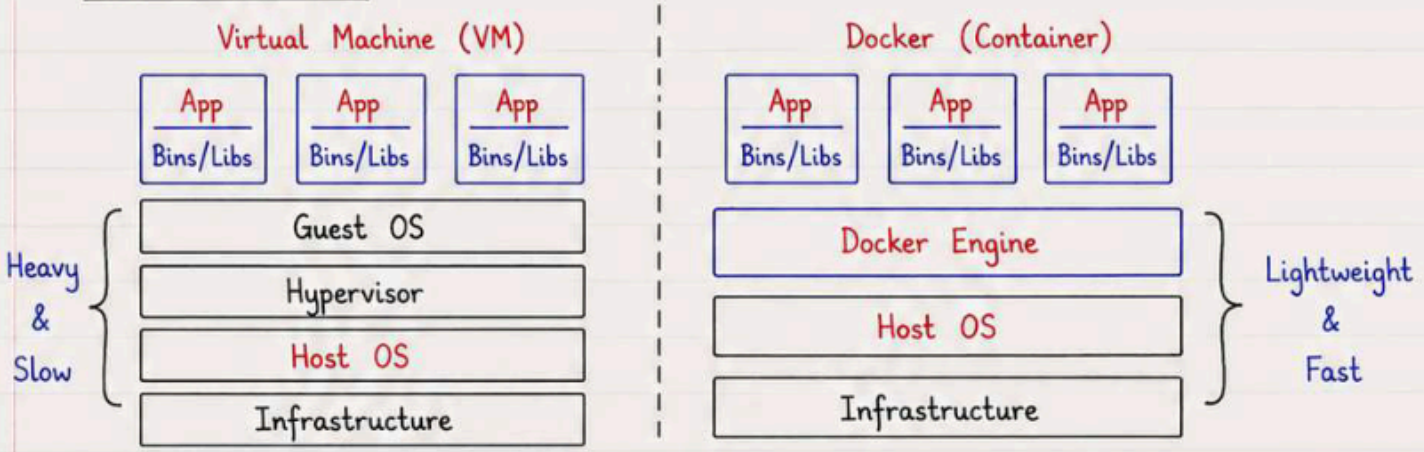
Before Docker, developers faced many issues while deploying applications.

- It works on my machine problem
- Different environments
- Heavy virtual machines
- Complex configuration
- Slow deployment

3. Benefits of Docker

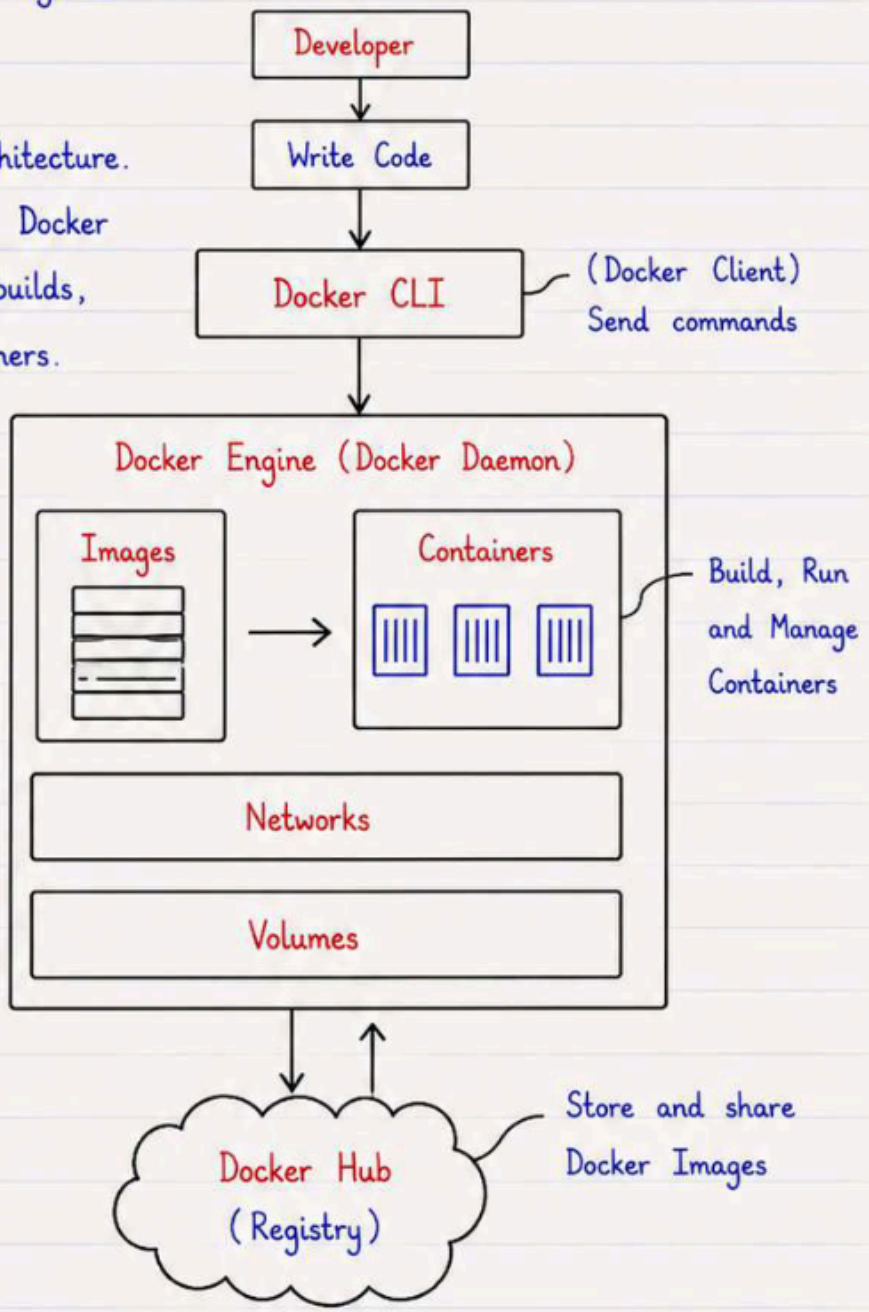
- Lightweight : Containers are smaller and use fewer resources.
- Portable : Build once and run anywhere.
- Consistent : Same environment everywhere.
- Fast Deployment : Start containers in seconds.
- Isolation : Each container runs isolated from others.
- Scalability : Easy to scale applications.

4. VM vs Docker



1. Docker Architecture

Docker follows a client-server architecture. Docker Client communicates with Docker Daemon (Docker Engine) which builds, runs and manages Docker containers.



Components :

1. Docker Client (CLI)

It is a command line tool. We use it to send commands to Docker Daemon.

2. Docker Daemon (Engine)

It is the core part of Docker. It builds images, runs containers, manages networks and volumes.

3. Docker Images

Read-only templates used to create containers.

4. Docker Containers

Running instances of images. They are lightweight, portable and isolated.

5. Docker Hub (Registry)

It is a cloud based registry service. We can push our images to Docker Hub or pull official images from it.

6. Networks

Used for communication between containers and with outside world.

Types : bridge, host, none, overlay.

7. Volumes

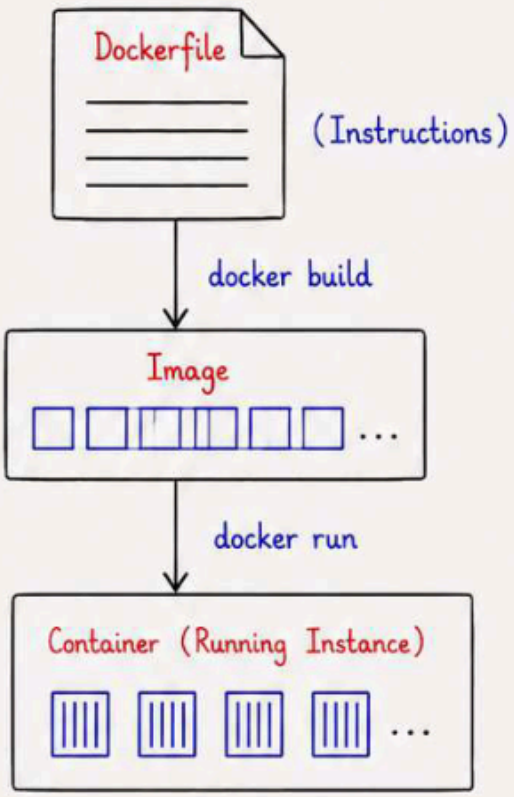
Used to store data generated by containers. Data in volumes persists even if container is removed.

How it works ?

We give commands through Docker CLI → It talks to Docker Daemon → Daemon pulls images from Docker Hub (if not present) → Creates and runs containers → Manages Networks and Volumes.

1. Image vs Container vs Dockerfile

Docker uses images to create containers. Image is a blueprint (read-only template) and container is a running instance of it. Dockerfile is a text file which has instructions to build an image.

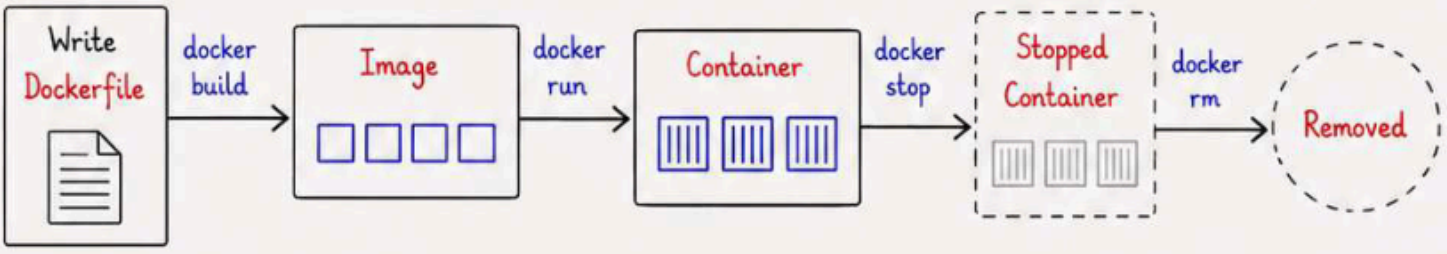


2. Image vs Container

Feature	Image	Container
Definition	Blueprint or template (read-only)	Running instance of image
State	Static	Dynamic
Change	Cannot be changed	Can be changed
Life	Permanent until deleted	Temporary, Can be stopped / removed
Size	Larger	Smaller
Purpose	To create containers	To run applications
Examples	ubuntu:22.04 mysql:8, nginx:latest	Running nginx, mysql container

3. Docker Lifecycle

A simple flow of how Dockerfile, Image and Container work together.

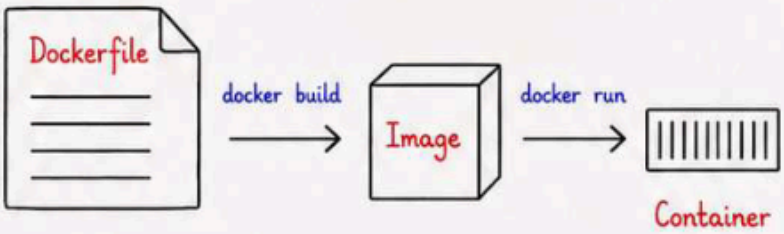


4. Key Points

- Image is built once and can be used to create multiple containers.
- Containers are isolated, portable and lightweight.
- Docker ensures "Build once, Run anywhere".
- Containers share the host OS kernel, that's why they are lightweight.

1. Dockerfile Explained

A Dockerfile is a text file that contains a set of instructions to build a Docker image automatically.



2. Sample Dockerfile

```
FROM openjdk:17-jdk-slim          # Base image
WORKDIR /app                      # Set working directory
COPY app.jar .                    # Copy file to container
ENV APP_ENV=production            # Set environment variable
EXPOSE 8080                       # Expose port
CMD ["java", "-jar", "app.jar"]   # Command to run app
```

> **Note :**
> Each instruction creates a layer in the image.
> If a layer doesn't change, Docker uses cache to build faster.

3. Important Instructions

- **FROM** : Specifies the base image to start with.
- **WORKDIR** : Sets the working directory inside the container.
- **COPY** : Copies files or directories from host to container.
- **ADD** : Similar to COPY, also supports URLs and auto-extracts .tar files.
- **ENV** : Sets environment variables inside the container.
- **EXPOSE** : Informs Docker that the container listens on the given port.
- **CMD** : Specifies the default command to run when container starts.
- **ENTRYPOINT** : Configures a container that will run as an executable. It allows the container to run as if it was that executable.

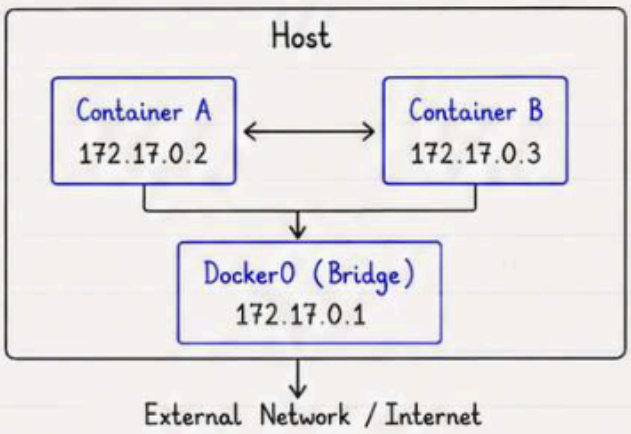
4. CMD vs ENTRYPOINT

CMD	ENTRYPOINT
• Provides default command and arguments. • Can be overridden when running container. • Only one CMD instruction is respected. • Used mostly for defining default behavior.	• Configures the container to run as executable. • Cannot be overridden easily. • Only one ENTRYPOINT instruction is respected. • Used when the container must always run a specific program.

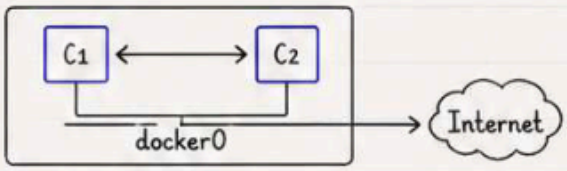
1. Docker Networking

Docker containers can communicate with each other and with outside world using networks.

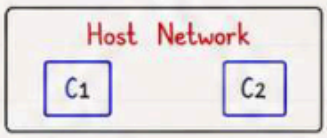
Types of Networks :



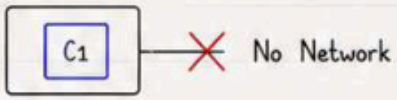
- **Bridge** (default) : Containers on the same bridge network can communicate with each other. Containers can access outside network.



- **Host** : Container shares the host network. No isolation at network level.



- **None** : Container has no network access. Useful for complete isolation.



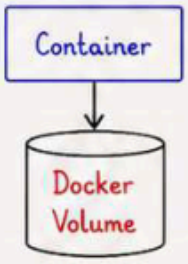
- **Overlay** : Used to connect containers across multiple Docker hosts (Swarm mode).



2. Storage in Docker

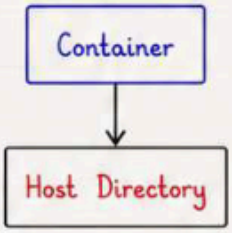
Containers are ephemeral. When a container is removed, its data is lost.
To persist data we use Volumes or Bind Mounts.

Volumes



- Managed by Docker.
- Stored in Docker host (/var/lib/docker/volumes).
- Platform independent.
- Recommended for production.

Bind Mounts



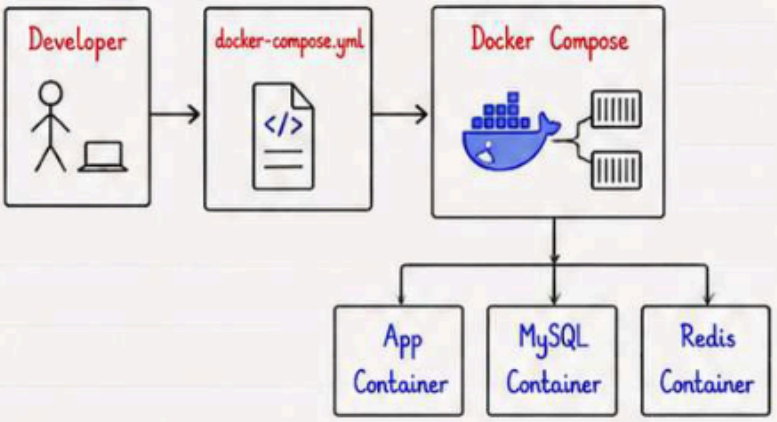
- Uses a directory from the host.
- Platform dependent.
- Good for development (code changes reflect immediately).

Feature	Volume	Bind Mount
Managed by	Docker	Host
Location	Inside Docker host	Anywhere on host system
Portability	High	Lower (depends on host path)
Backup	Easy (docker volume backup)	Manual
Best for	Production	Development

1. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. With a single YAML file, we can configure all the services, networks and volumes required for our app.
It makes it easy to start, stop and manage the entire application.

How it works ?



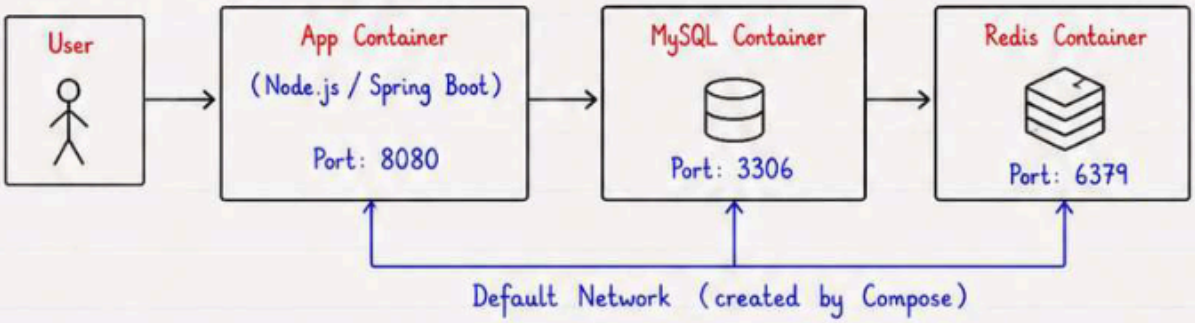
2. docker-compose.yml (Example)

```yaml
version: '3.8'                    # Compose file version
services:                         # All services
  app:                            # Service name
    image: myapp:latest           # Image to use
    ports:                        # Port mapping host:container
      - "8080:8080"
    depends_on:                   # Start mysql before app
      - mysql
  mysql:                          # Another service
    image: mysql:8                # MySQL image
    environment:                  # Environment variable
      MYSQL_ROOT_PASSWORD: root
    volumes:                      # Persist data in volume
      - db_data:/var/lib/mysql
  redis:                          # Another service
    image: redis:latest           # Redis image
volumes:
  db_data:                        # Named volume
```

3. Important Commands

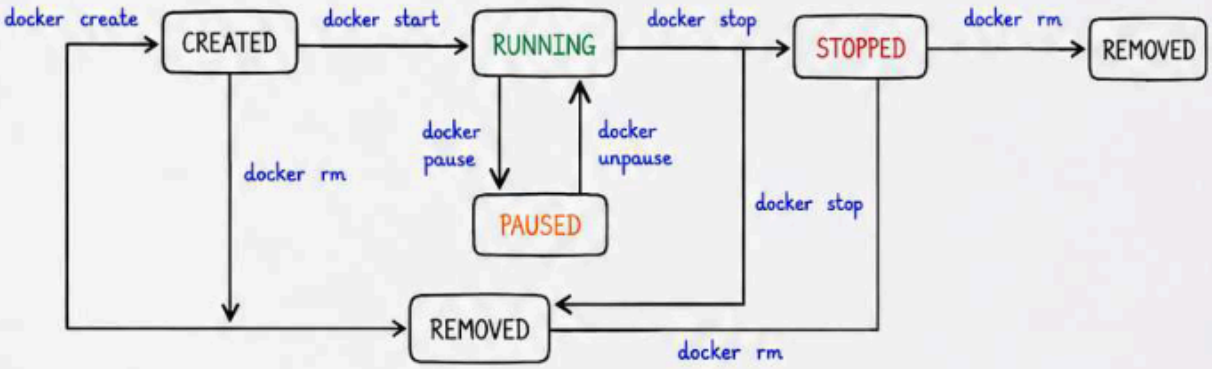
- docker compose up
  → Build and start all services defined in the file.
- docker compose up -d
  → Start in detached mode (in background).
- docker compose down
  → Stop and remove all containers, networks.
- docker compose logs
  → View logs of all services.
- docker compose ps
  → List all running services.
- docker compose build
  → Build or rebuild services.

4. Multi-container Application Example



1. Docker Container Lifecycle

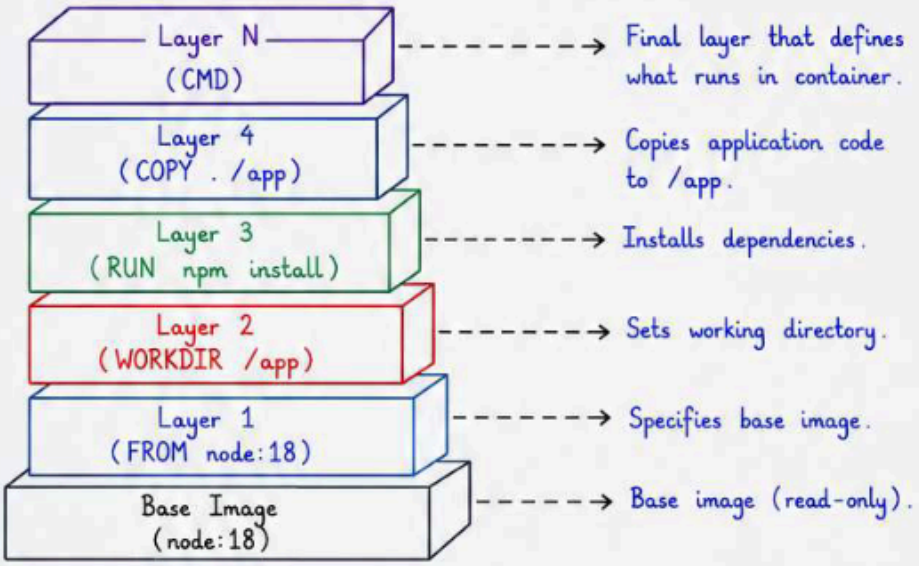
A container goes through different states during its lifetime.



- **Created** : Container is created but not started.
- **Running** : Container is up and running.
- **Paused** : Container is running but all processes are paused.
- **Stopped** : Container is stopped but still exists.
- **Removed** : Container is deleted and cannot be recovered.

2. Docker Image Layers

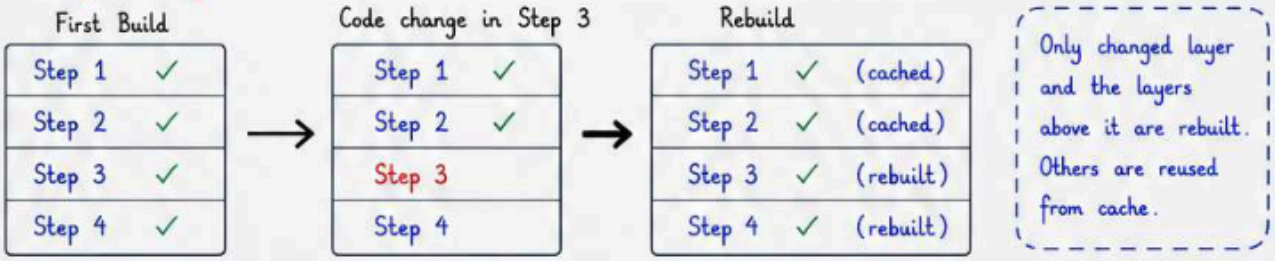
Docker images are built using a layered architecture.
Each instruction in the Dockerfile adds a new layer.



Notes :

- Layers are read-only.
- Only the top writable layer changes when container runs.
- Docker reuses cached layers to build images faster.
- Smaller layers = better performance.

How Layer Caching Works ?



Only changed layer and the layers above it are rebuilt. Others are reused from cache.

– Page 8 – Docker Commands Cheat Sheet –

Here are the most commonly used Docker commands every developer / tester should know.

Command	Description	Example
docker --version	Check Docker version	docker --version
docker info	Display system-wide information	docker info
docker login	Login to Docker registry	docker login
docker logout	Logout from Docker registry	docker logout
docker search <image>	Search image on Docker Hub	docker search nginx
docker pull <image>	Download image from registry	docker pull nginx:latest
docker images	List all downloaded images	docker images
docker rmi <image-id>	Remove an image	docker rmi nginx
docker build -t <name> .	Build image from Dockerfile	docker build -t myapp:1.0 .
docker run <image>	Run a container	docker run nginx
docker run -d <image>	Run container in detached mode	docker run -d nginx
docker ps	List running containers	docker ps
docker ps -a	List all containers (running + stopped)	docker ps -a
docker start <container-id>	Start a stopped container	docker start a1b2c3d4
docker stop <container-id>	Stop a running container	docker stop a1b2c3d4
docker restart <container-id>	Restart a container	docker restart a1b2c3d4
docker rm <container-id>	Remove a stopped container	docker rm a1b2c3d4
docker rm -f <container-id>	Force remove a running container	docker rm -f a1b2c3d4
docker logs <container-id>	View logs of a container	docker logs a1b2c3d4
docker exec -it <container-id> bash	Access container shell	docker exec -it a1b2c3d4 bash
docker inspect <container-id>	Inspect container details	docker inspect a1b2c3d4
docker cp <src> <container>:/dest	Copy files/folders to container	docker cp app.py a1b2c3d4:/app
docker cp <container>:/src <dest>	Copy files/folders from container	docker cp a1b2c3d4:/app /home/user
docker network ls	List all networks	docker network ls
docker volume ls	List all volumes	docker volume ls
docker system prune -a	Remove unused data (containers, images, networks, volumes)	docker system prune -a